GREEEN tallgrass prairie restoration: Seeded richness ~ Species pool size (NW Station)

MB

2025-10-28

Contents

Preparation **1**

Statistics **2**
Data exploration . . . 2
Means and deviations . . . 2
Graphs of raw data (Step 2, 6, 7) . . . 4
Outliers, zero-inflation, transformations? (Step 1, 3, 4) . . . 6
Check collinearity part 1 (Step 5) . . . 6
Models . . . 6
Model check . . . 7
DHARMa . . . 7
Check collinearity part 2 (Step 5) . . . 10
Model comparison . . . 11
R2 values . . . 11
AICc . . . 11
Predicted values . . . 11
Summary table . . . 11
Forest plot . . . 13
Effect sizes . . . 14

Session info **16**

> **NOTE:** To compare different models, you only have to change the models in the section 'Load models'

Preparation

Protocol of data exploration (Steps 1-8) used from Zuur et al. (2010) Methods Ecol Evol DOI: 10.1111/2041-210X.12577

```
library(here)
library(tidyverse)
library(ggbeeswarm)
library(patchwork)
library(lme4)
library(glmmTMB)
```

```r
library(DHARMa)
library(emmeans)
```

Packages

```r
sites <- read_csv(
  here("data", "processed", "data_processed_sites.csv"),
  col_names = TRUE, na = c("na", "NA", ""), col_types = cols(
    .default = "?",
    id_plot = "f",
    year = "f",
    seeding_time = col_factor(
      levels = c("unseeded", "fall", "spring"), ordered = FALSE
      ),
    seeded_pool = col_factor(
      levels = c("0", "6", "12", "18", "33"), ordered = TRUE
      ),
    treatment_id = "f",
    richness_type = "f"
  )
) %>%
  filter(
    site == "NW Station",
    richness_type == "seeded_richness",
    !(treatment_id %in% c("2", "4")),
    !(year == 2016)
  ) %>%
  mutate(y = richness_1qm + richness_25qm)
```

Load data

Statistics

Data exploration

Means and deviations

```r
Rmisc::CI(sites$y, ci = .95)
```

```
##    upper     mean    lower
## 1.852948 1.611111 1.369275
```

```r
median(sites$y)
```

```
## [1] 1
```

```r
sd(sites$y)
```

```
## [1] 1.468128
```

```r
quantile(sites$y, probs = c(0.05, 0.95), na.rm = TRUE)
```

```
##  5% 95%
##   0   4
```

```r
sites %>% count(site, year)
```

```
## # A tibble: 3 x 3
##   site       year      n
##   <chr>      <fct> <int>
## 1 NW Station 2015     48
## 2 NW Station 2017     48
## 3 NW Station 2018     48
```

```r
sites %>% count(seeded_pool)
```

```
## # A tibble: 4 x 2
##   seeded_pool     n
##   <ord>       <int>
## 1 6              36
## 2 12             36
## 3 18             36
## 4 33             36
```

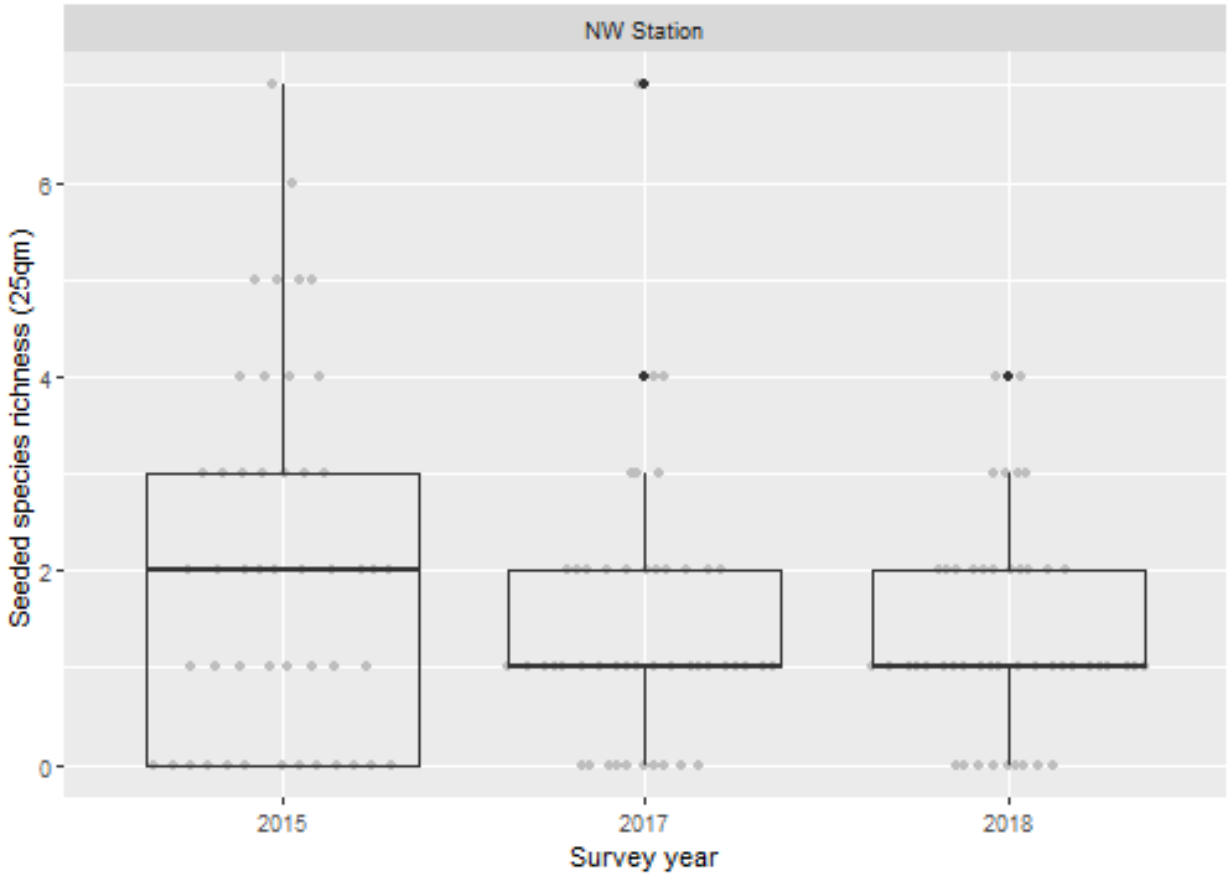
```r
sites %>% count(seeding_time)
```

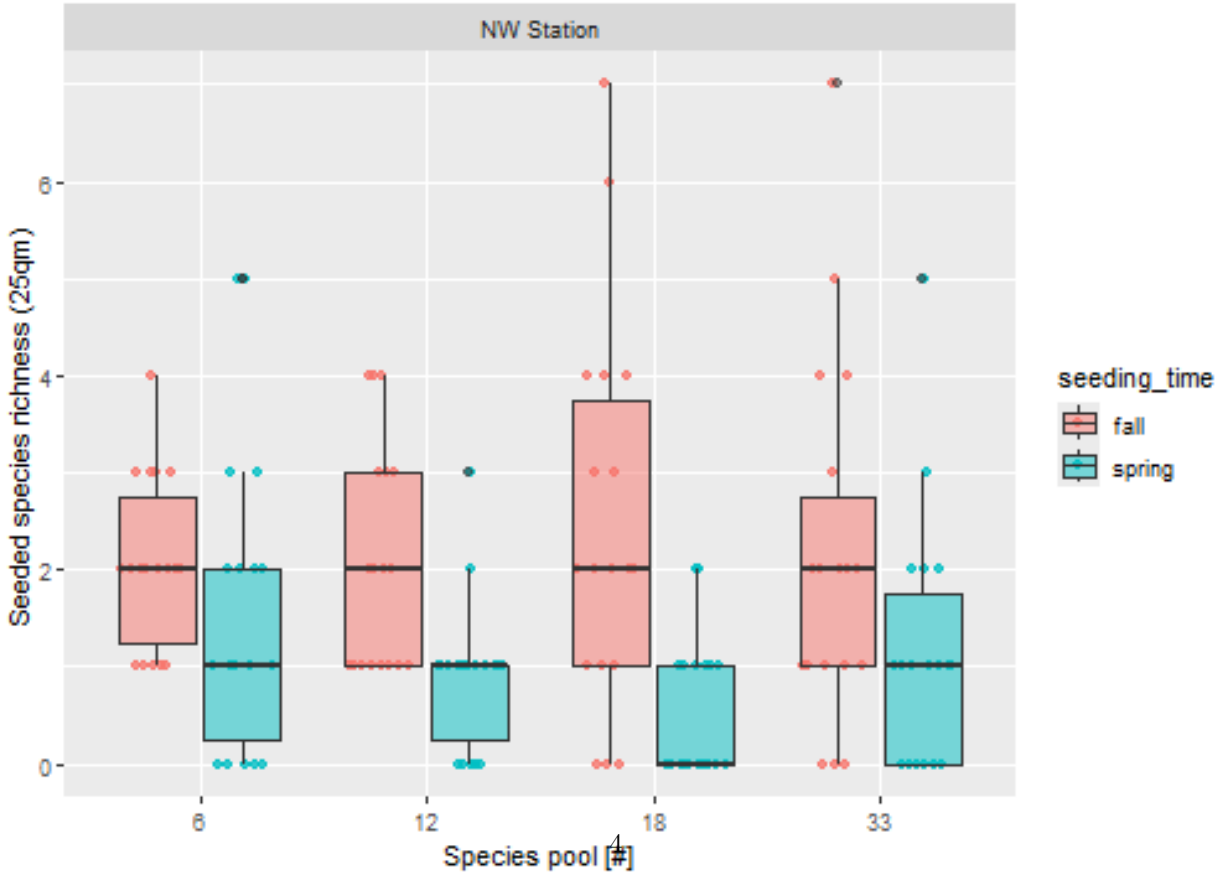
```
## # A tibble: 2 x 2
##   seeding_time     n
##   <fct>        <int>
## 1 fall            72
## 2 spring          72
```

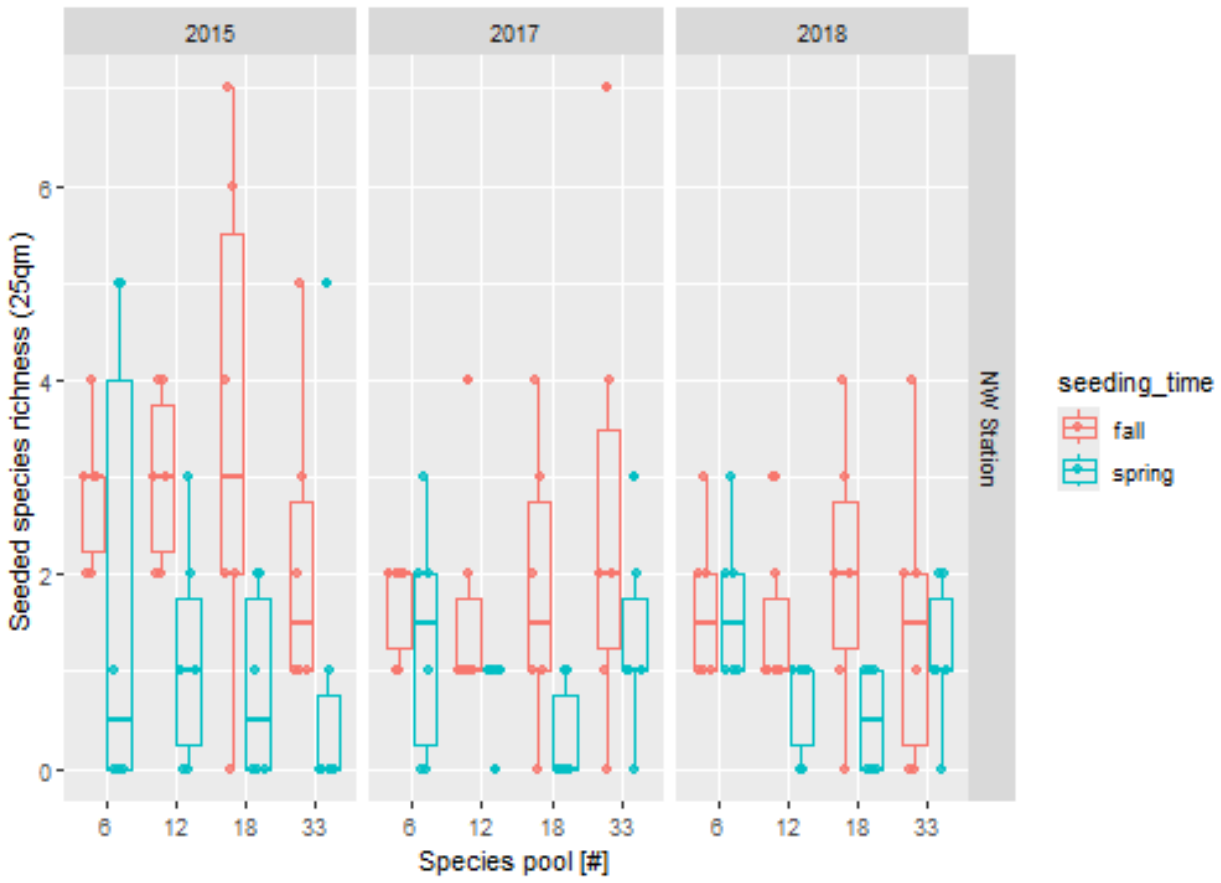
```r
sites %>% count(herbicide)
```

```
## # A tibble: 2 x 2
##   herbicide     n
##       <dbl> <int>
## 1         0    72
## 2         1    72
```

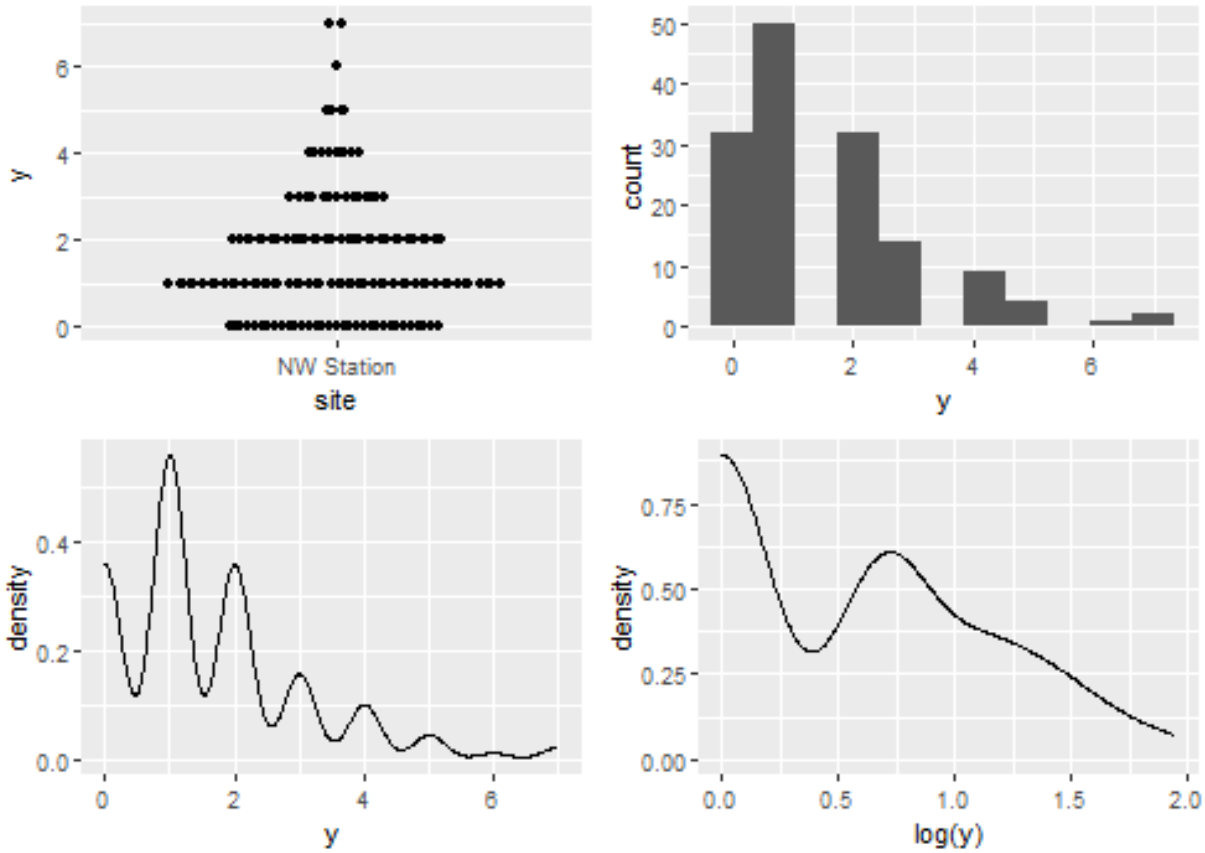
Graphs of raw data (Step 2, 6, 7)






2015
2017
2018
NW Station
Seeded species richness (25qm)
0
2
4
6
6
12
18
33
Species pool [#]
seeding_time
fall
spring

Outliers, zero-inflation, transformations? (Step 1, 3, 4)



Check collinearity part 1 (Step 5)

Exclude r > 0.7 Dormann et al. 2013 Ecography DOI: 10.1111/j.1600-0587.2012.07348.x

```
# sites %>%
#   select(where(is.numeric), -y, -starts_with("cwm.")) %>%
#   GGally::ggpairs(
#     lower = list(continuous = "smooth_loess")
#     ) +
#   theme(strip.text = element_text(size = 7))

# -> no continuous variables
```

Models

NOTE: Only here you have to modify the script to compare other models

```
load(file = here("outputs", "models", "model_richness_pool_nws_1.Rdata"))
m_1 <- m1
# m_2 <- m2
```

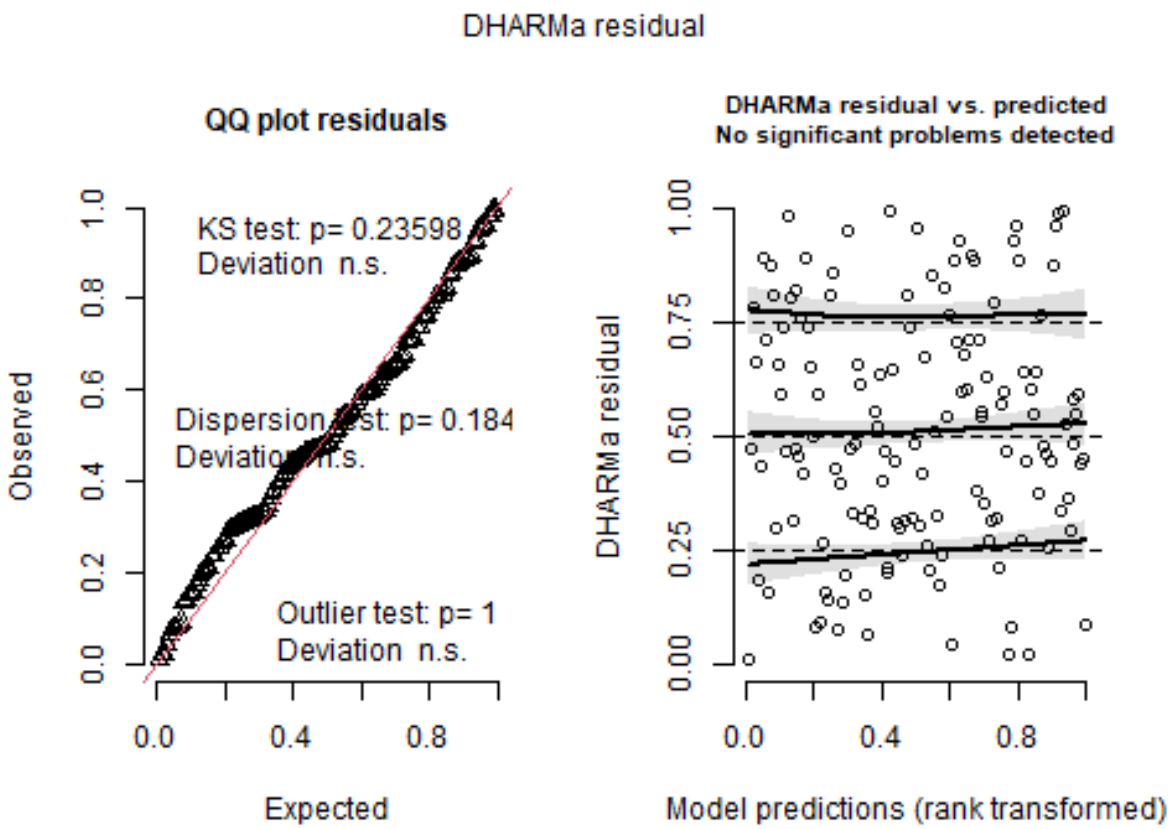
```
formula(m_1)
## y ~ seeded_pool * seeding_time * year + water_cap
class(m_1)
## [1] "glm" "lm"
```

```
# formula(m_2)
# class(m_2)
# m_1@call
# m_2@call
```

Model check

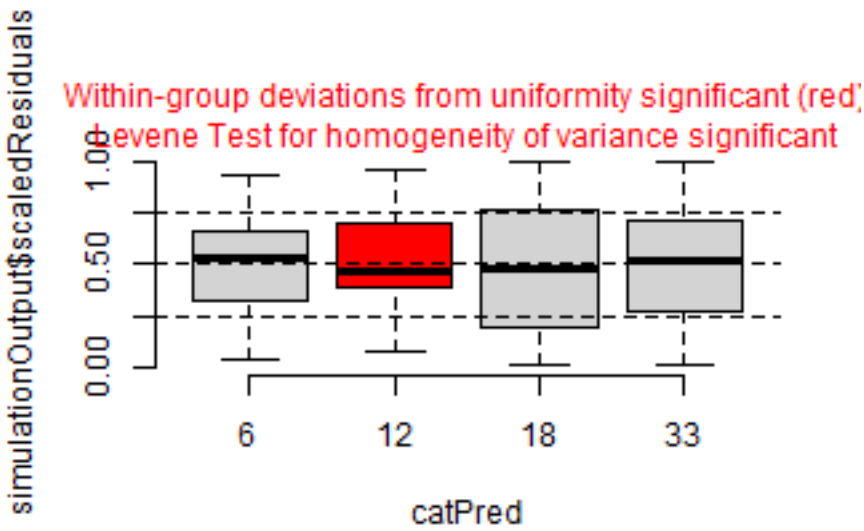
DHARMa

```
simulation_output_1 <- simulateResiduals(m_1, plot = TRUE)
```

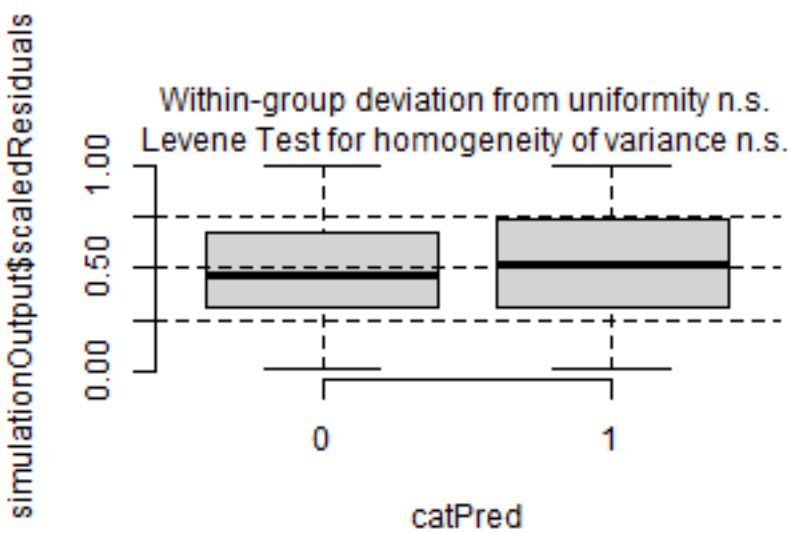



```
# simulation_output_2 <- simulateResiduals(m_2, plot = TRUE)
```

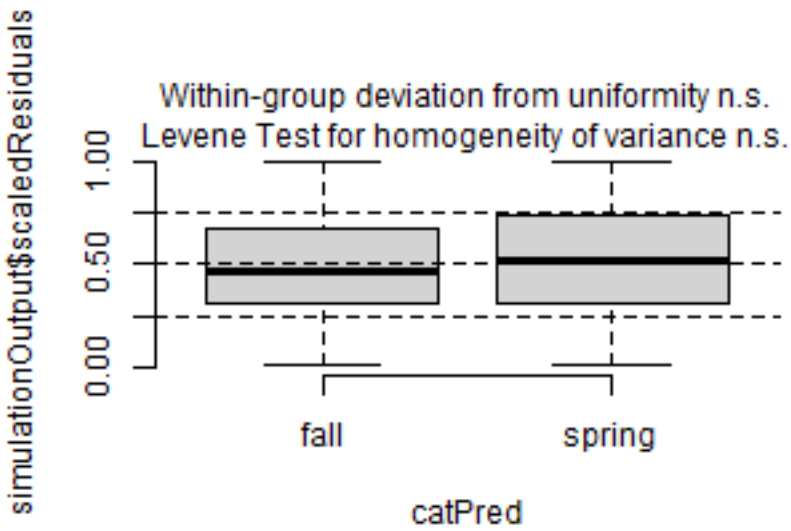
```
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(seeded_pool))
  )
```



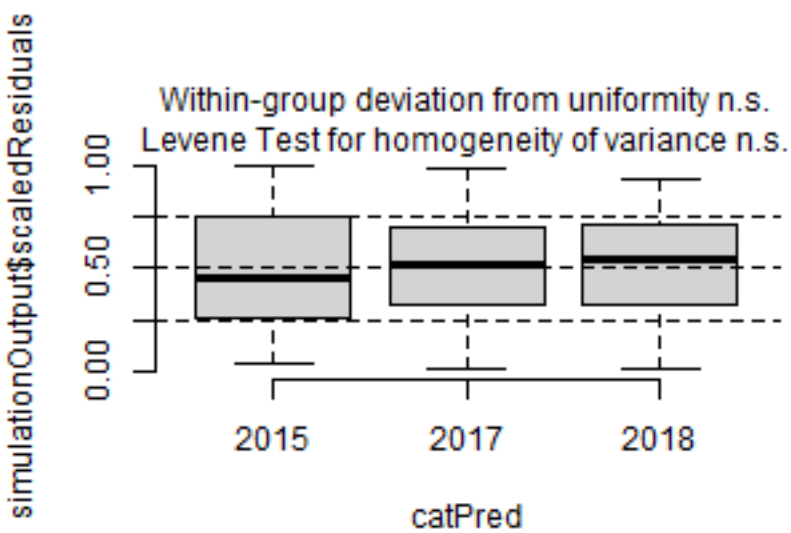

```
# plotResiduals(simulation_output_2$scaledResiduals, factor(sites$seeded_pool))
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(herbicide))
  )
```



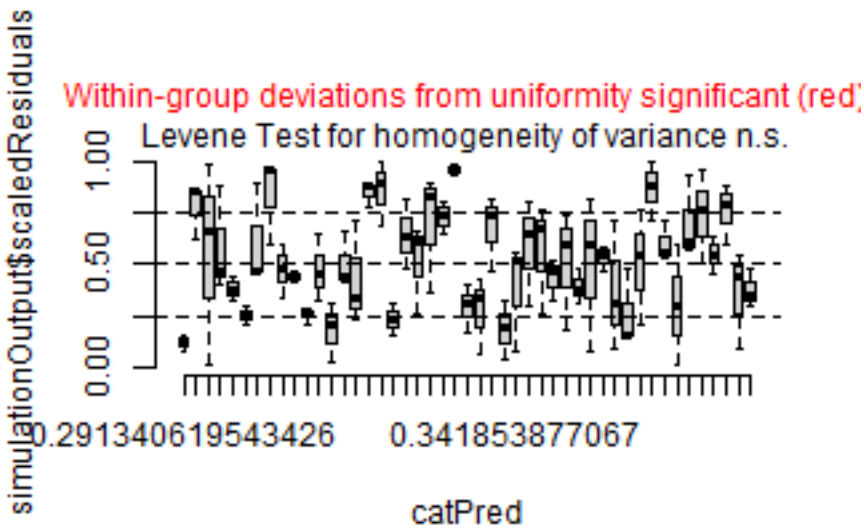

```
# plotResiduals(simulation_output_2$scaledResiduals, sites$herbicide)
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(seeding_time))
  )
```




```r
# plotResiduals(simulation_output_2$scaledResiduals, sites$seeding_time)
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(year))
  )
```

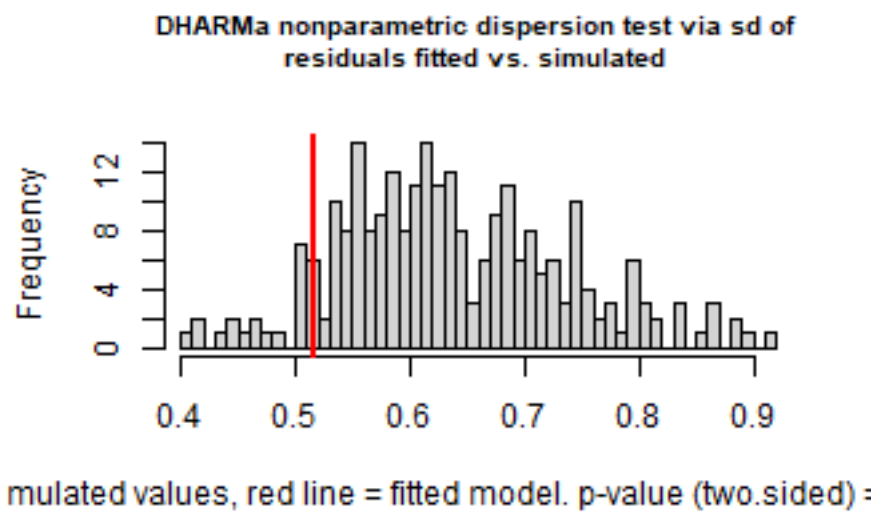



```r
# plotResiduals(simulation_output_2$scaledResiduals, sites$year)
plotResiduals(
  simulation_output_1$scaledResiduals,
  factor(sites %>% filter(!(is.na(water_cap))) %>%  pull(water_cap))
  )
```




```
# plotResiduals(simulation_output_2$scaledResiduals, sites$water_cap)
```

```
testDispersion(simulation_output_1)
```



Test for overdispersion

```
##
##  DHARMa nonparametric dispersion test via sd of residuals fitted vs.
##  simulated
##
## data:  simulationOutput
## dispersion = 0.80936, p-value = 0.184
## alternative hypothesis: two.sided
# testDispersion(simulation_output_2)
```

Check collinearity part 2 (Step 5)

Remove VIF > 3 or > 10 Zuur et al. 2010 Methods Ecol Evol DOI: 10.1111/j.2041-210X.2009.00001.x

```r
car::vif(m_1)
```

```
## there are higher-order terms (interactions) in this model
## consider setting type = 'predictor'; see ?vif
```

```
##                                     GVIF Df GVIF^(1/(2*Df))
## seeded_pool                    40.424166  3        1.852565
## seeding_time                    2.709980  1        1.646202
## year                            2.375919  2        1.241532
## water_cap                       1.227468  1        1.107912
## seeded_pool:seeding_time       51.623509  3        1.929632
## seeded_pool:year              155.773703  6        1.523028
## seeding_time:year               6.227803  2        1.579733
## seeded_pool:seeding_time:year 241.723821  6        1.579829
```

```r
# car::vif(m_2)
```

Model comparison

R2 values

```r
MuMIn::r.squaredGLMM(m_1)
## Warning: the null model is only correct if all the variables it uses are identical
## to those used in fitting the original model.
##                 R2m       R2c
## delta     0.3836168 0.3836168
## lognormal 0.4444544 0.4444544
## trigamma  0.3122905 0.3122905
# MuMIn::r.squaredGLMM(m_2)
```

AICc

Use AICc and not AIC since ratio n/K < 40 Burnahm & Anderson 2002 p. 66 ISBN: 978-0-387-95364-9

```r
# MuMIn::AICc(m_1, m_2) %>%
#   arrange(AICc)
```

Predicted values

Summary table

```r
car::Anova(m_1, type = 3)
```

```
## Analysis of Deviance Table (Type III tests)
##
## Response: y
##                               LR Chisq Df Pr(>Chisq)
## seeded_pool                     1.9393  3    0.58510
## seeding_time                   20.6897  1  5.401e-06 ***
## year                            8.2016  2    0.01656 *
## water_cap                      20.3039  1  6.607e-06 ***
## seeded_pool:seeding_time        3.6771  3    0.29851
## seeded_pool:year                3.6538  6    0.72341
## seeding_time:year               0.6724  2    0.71450
## seeded_pool:seeding_time:year   0.6886  6    0.99473
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

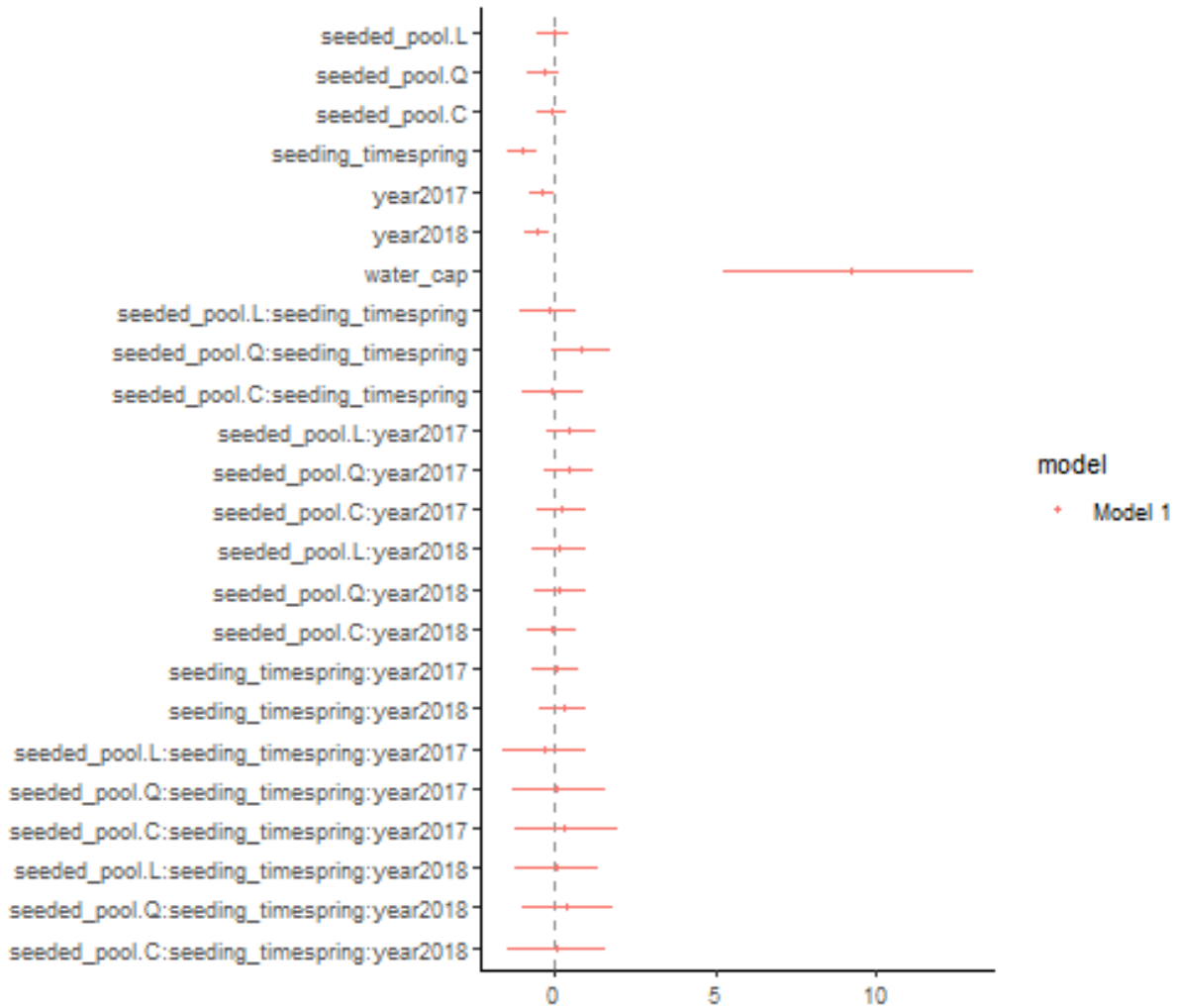
```r
summary(m_1)
```

```
##
## Call:
## glm(formula = y ~ seeded_pool * seeding_time * year + water_cap,
##     family = poisson(link = "log"), data = sites)
##
## Coefficients:
##                                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                              -2.09031    0.68940  -3.032  0.00243
## seeded_pool.L                            -0.02297    0.25837  -0.089  0.92916
## seeded_pool.Q                            -0.33346    0.24452  -1.364  0.17265
## seeded_pool.C                            -0.04955    0.23353  -0.212  0.83195
## seeding_timespring                       -0.99613    0.23131  -4.306 1.66e-05
## year2017                                 -0.38935    0.19241  -2.024  0.04301
## year2018                                 -0.53778    0.20070  -2.680  0.00737
## water_cap                                 9.21494    1.97052   4.676 2.92e-06
## seeded_pool.L:seeding_timespring         -0.19888    0.44670  -0.445  0.65617
## seeded_pool.Q:seeding_timespring          0.80002    0.45920   1.742  0.08147
## seeded_pool.C:seeding_timespring         -0.07096    0.48091  -0.148  0.88269
## seeded_pool.L:year2017                    0.48209    0.38590   1.249  0.21157
## seeded_pool.Q:year2017                    0.45571    0.38481   1.184  0.23632
## seeded_pool.C:year2017                    0.20455    0.38373   0.533  0.59399
## seeded_pool.L:year2018                    0.13914    0.41348   0.337  0.73649
## seeded_pool.Q:year2018                    0.17720    0.40140   0.441  0.65887
## seeded_pool.C:year2018                   -0.05315    0.38894  -0.137  0.89131
## seeding_timespring:year2017               0.03508    0.36748   0.095  0.92394
## seeding_timespring:year2018               0.28488    0.35978   0.792  0.42847
## seeded_pool.L:seeding_timespring:year2017 -0.29471    0.66144  -0.446  0.65592
## seeded_pool.Q:seeding_timespring:year2017  0.08852    0.73495   0.120  0.90413
## seeded_pool.C:seeding_timespring:year2017  0.29008    0.80175   0.362  0.71750
## seeded_pool.L:seeding_timespring:year2018  0.03912    0.66519   0.059  0.95311
## seeded_pool.Q:seeding_timespring:year2018  0.38744    0.71957   0.538  0.59028
## seeded_pool.C:seeding_timespring:year2018  0.07620    0.77012   0.099  0.92118
##
## (Intercept)                              **
## seeded_pool.L
## seeded_pool.Q
## seeded_pool.C
## seeding_timespring                       ***
## year2017                                 *
## year2018                                 **
## water_cap                                ***
## seeded_pool.L:seeding_timespring
## seeded_pool.Q:seeding_timespring         .
## seeded_pool.C:seeding_timespring
## seeded_pool.L:year2017
## seeded_pool.Q:year2017
## seeded_pool.C:year2017
## seeded_pool.L:year2018
## seeded_pool.Q:year2018
## seeded_pool.C:year2018
```

```
## seeding_timespring:year2017
## seeding_timespring:year2018
## seeded_pool.L:seeding_timespring:year2017
## seeded_pool.Q:seeding_timespring:year2017
## seeded_pool.C:seeding_timespring:year2017
## seeded_pool.L:seeding_timespring:year2018
## seeded_pool.Q:seeding_timespring:year2018
## seeded_pool.C:seeding_timespring:year2018
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 193.11  on 140  degrees of freedom
## Residual deviance: 120.12  on 116  degrees of freedom
##   (3 Beobachtungen als fehlend gelöscht)
## AIC: 448.23
##
## Number of Fisher Scoring iterations: 6
```

Forest plot

```r
dotwhisker::dwplot(
  list(m_1),
  ci = 0.95,
  show_intercept = FALSE,
  vline = geom_vline(xintercept = 0, colour = "grey60", linetype = 2)) +
  theme_classic()
```



Effect sizes

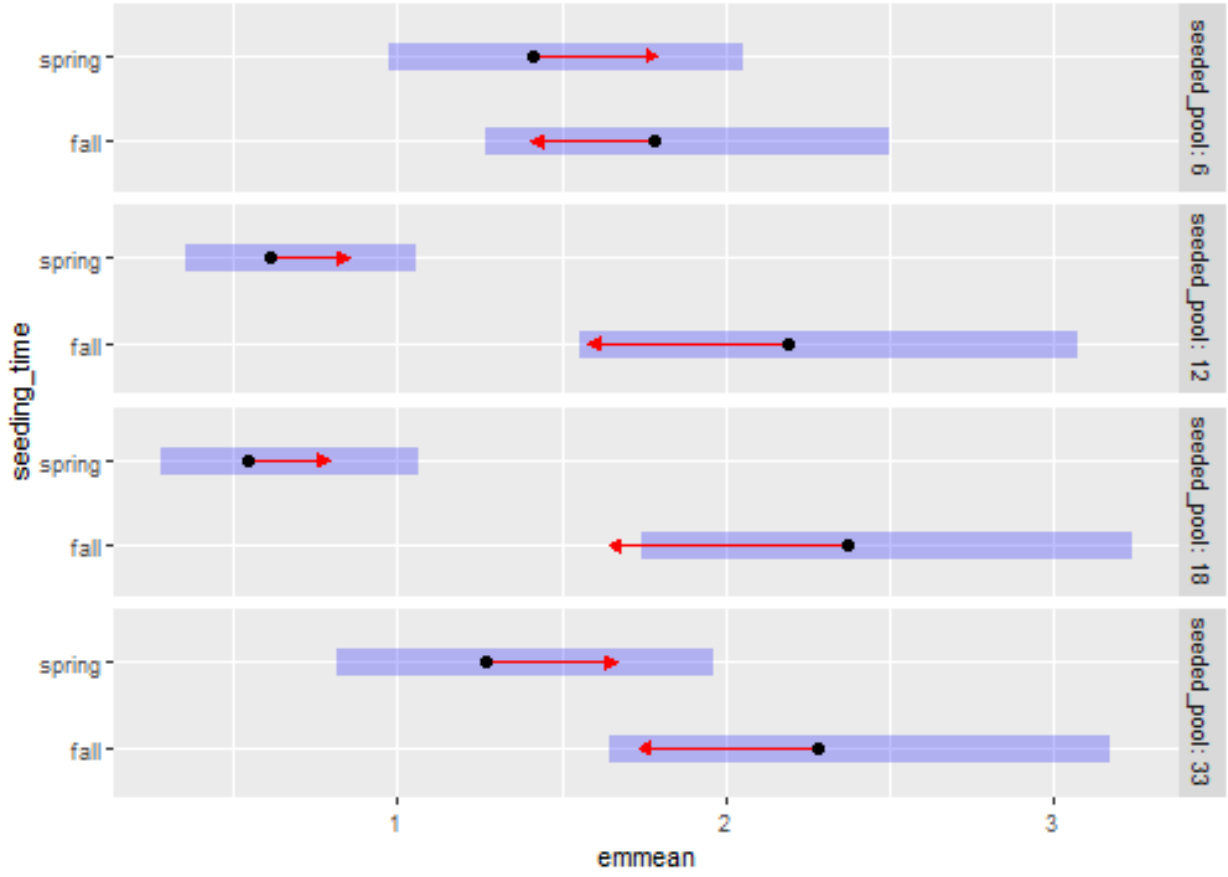
Effect sizes of chosen model just to get exact values of means etc. if necessary.

```
(emm <- emmeans(
  m_1,
  revpairwise ~ seeding_time | seeded_pool,
  type = "response"
  ))
```

```
## $emmeans
## seeded_pool = 6:
##  seeding_time  rate    SE  df asymp.LCL asymp.UCL
##  fall         1.784 0.307 Inf     1.273      2.50
##  spring       1.416 0.269 Inf     0.975      2.06
##
## seeded_pool = 12:
##  seeding_time  rate    SE  df asymp.LCL asymp.UCL
##  fall         2.191 0.380 Inf     1.559      3.08
##  spring       0.615 0.169 Inf     0.358      1.06
```

```
##
## seeded_pool = 18:
##  seeding_time  rate    SE  df asymp.LCL asymp.UCL
##  fall         2.376 0.375 Inf     1.743      3.24
##  spring       0.548 0.186 Inf     0.282      1.07
##
## seeded_pool = 33:
##  seeding_time  rate    SE  df asymp.LCL asymp.UCL
##  fall         2.284 0.383 Inf     1.644      3.17
##  spring       1.267 0.284 Inf     0.816      1.97
##
## Results are averaged over the levels of: year
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
##
## $contrasts
## seeded_pool = 6:
##  contrast      ratio     SE  df null z.ratio p.value
##  spring / fall 0.794 0.2010 Inf    1  -0.914  0.3607
##
## seeded_pool = 12:
##  contrast      ratio     SE  df null z.ratio p.value
##  spring / fall 0.281 0.0932 Inf    1  -3.826  0.0001
##
## seeded_pool = 18:
##  contrast      ratio     SE  df null z.ratio p.value
##  spring / fall 0.231 0.0864 Inf    1  -3.918  0.0001
##
## seeded_pool = 33:
##  contrast      ratio     SE  df null z.ratio p.value
##  spring / fall 0.555 0.1560 Inf    1  -2.097  0.0360
##
## Results are averaged over the levels of: year
## Tests are performed on the log scale
```

```r
plot(emm, comparison = TRUE)
```



Session info

```
## R version 4.5.1 (2025-06-13 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=German_Germany.utf8  LC_CTYPE=German_Germany.utf8
## [3] LC_MONETARY=German_Germany.utf8 LC_NUMERIC=C
## [5] LC_TIME=German_Germany.utf8
##
## time zone: Europe/Berlin
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] emmeans_1.11.1   DHARMa_0.4.7     glmmTMB_1.1.12   lme4_1.1-37
##  [5] Matrix_1.7-3     patchwork_1.3.1  ggbeeswarm_0.7.2 lubridate_1.9.4
##  [9] forcats_1.0.0    stringr_1.5.1    dplyr_1.1.4      purrr_1.1.0
```

```
## [13] readr_2.1.5      tidyr_1.3.1      tibble_3.3.0     ggplot2_3.5.2
## [17] tidyverse_2.0.0  here_1.0.1
##
## loaded via a namespace (and not attached):
##  [1] Rdpack_2.6.4           gridExtra_2.3          sandwich_3.1-1
##  [4] rlang_1.1.6            magrittr_2.0.3         compiler_4.5.1
##  [7] mgcv_1.9-3             vctrs_0.6.5            pkgconfig_2.0.3
## [10] crayon_1.5.3           fastmap_1.2.0          backports_1.5.0
## [13] labeling_0.4.3         utf8_1.2.6             ggstance_0.3.7
## [16] promises_1.3.3         rmarkdown_2.29         tzdb_0.5.0
## [19] nloptr_2.2.1           bit_4.6.0              xfun_0.52
## [22] later_1.4.2            parallel_4.5.1         R6_2.6.1
## [25] gap.datasets_0.0.6     stringi_1.8.7          qgam_2.0.0
## [28] RColorBrewer_1.1-3     car_3.1-3              boot_1.3-31
## [31] numDeriv_2016.8-1.1    estimability_1.5.1     Rcpp_1.1.0
## [34] iterators_1.0.14       knitr_1.50             zoo_1.8-14
## [37] parameters_0.27.0      httpuv_1.6.16          splines_4.5.1
## [40] timechange_0.3.0       tidyselect_1.2.1       rstudioapi_0.17.1
## [43] abind_1.4-8            yaml_2.3.10            MuMIn_1.48.11
## [46] doParallel_1.0.17      TMB_1.9.17             codetools_0.2-20
## [49] lattice_0.22-7         plyr_1.8.9             shiny_1.11.1
## [52] withr_3.0.2            bayestestR_0.16.1      coda_0.19-4.1
## [55] evaluate_1.0.4         marginaleffects_0.28.0 pillar_1.11.0
## [58] gap_1.6                carData_3.0-5          foreach_1.5.2
## [61] stats4_4.5.1           reformulas_0.4.1       insight_1.3.1
## [64] generics_0.1.4         vroom_1.6.5            rprojroot_2.1.1
## [67] hms_1.1.3              scales_1.4.0           minqa_1.2.8
## [70] xtable_1.8-4           glue_1.8.0             tools_4.5.1
## [73] data.table_1.17.8      mvtnorm_1.3-3          grid_4.5.1
## [76] rbibutils_2.3          datawizard_1.1.0       nlme_3.1-168
## [79] Rmisc_1.5.1            performance_0.15.0     beeswarm_0.4.0
## [82] vipor_0.4.7            Formula_1.2-5          cli_3.6.5
## [85] gtable_0.3.6           digest_0.6.37          farver_2.1.2
## [88] htmltools_0.5.8.1      lifecycle_1.0.4        mime_0.13
## [91] bit64_4.6.0-1          dotwhisker_0.8.4       MASS_7.3-65
```